第 0-1 讲 • 打开黑框：终端与命令行

> **模块 0 • 热身** ｜ 建议用时 **40 分钟**（阅读 20 + 动手 20）
> 前置要求：无。这一讲假设你从没打开过终端。

本节目标

读完并做完这一讲，你能：

1. **打开终端**，并说出它还叫什么名字（命令行、CMD、PowerShell、shell——是同一类东西的不同叫法）。
2. **在终端里移动位置**：知道自己"现在站在哪个文件夹"，并能走到别的文件夹。
3. **看清文件夹里有什么**，包括被隐藏的文件。
4. **用一句命令确认自己的电脑环境**：Node.js 装没装、什么版本。
5. 解释一句最关键的话：**"图形界面是个壳，底下一直在跑命令。"**

如果你读完只有一件事记住了，希望是第 5 条。它会让后面 23 讲都变简单。

一、为什么必须学这个

1.1 一个真实的场景

你在写这份讲义的作者那里问："帮我看看项目能不能跑起来。" 对方回你：

```
cd C:\Users\34123\Desktop\记账簿
npm run dev
```

如果你不认识这两行字，你只能回"这是什么意思"。

但如果你认识，你会知道：

- 第一行是**走到记账簿这个文件夹去**；
- 第二行是**让这个项目跑起来。**

这就是全部的区别。 编程世界里大量的交流是用命令完成的，不是因为程序员喜欢黑框，而是因为**命令比截图更精确、更省事、可以复制。**

1.2 终端不是"高级功能"，它是最原始的那一层

这里有个关键认知，值得你花两分钟读：

电脑的图形界面（你能用鼠标点的那些窗口、按钮）是后来才有的。 最早使用电脑，就是打字下命令。后来有人觉得太难用，才在上面盖了一层"壳"，让你能点着用。

所以今天你双击一个图标，电脑内部做的事是：

```
你双击图标   →   Windows 把"双击"翻译成一条命令   →   执行那个程序
   点鼠标              翻译                              真正干活
```

图形界面是壳，命令是里子。 很多软件（尤其是开发用的工具）只提供命令这一层，因为它们不需要照顾"好不好点"，只需要准确。记账簿的启动、打包、测试，全部是命令——你之前看到我跑的那些 `npm run ...`，就是这个。

1.3 学会它对你有什么实际好处

- 你能自己把项目跑起来，不用每次都问别人；
- 出错时你能把**文字**发给别人（截图别人不好搜，文字能直接搜）；
- 你能看懂我给你的所有指令，并且知道**它在干什么**，而不是照着抄；
- 模块 6 打包发布时，你要靠命令出安装包。

二、终端到底是什么

2.1 现实比喻：一个只会听话的柜台

想象一个办事柜台。它不说话、不猜你的意思，**你写什么它做什么，一字不差。**

- 你写"列出这个文件夹里的东西"，它就列出来。
- 你写"走到上一层"，它就往上走。
- 你写错了，它不会说"你是不是想写……"，它只会说"你说的这个我不认识"。

这个柜台有三个特点，理解了就不会怕它：

特点	意味着什么
它记不住上次	每次关掉再开，位置回到最初的默认位置（通常是你的用户目录）
它只认当前所在位置	你说"打开 a.txt"，它只会在**你当前站的地方**找，不会全硬盘搜
它一字不差	多一个空格、少一个引号，意思可能就变了

第二条特别重要：**终端永远"站在"一个文件夹里。**这就是为什么必须有"cd"（走目录）这个动作。

2.2 术语对齐

你听到的说法	其实是同一个东西
终端、Terminal	那个窗口本身
命令行、Command Line	你输入命令这种**方式**
CMD、命令提示符	Windows 上老的那个具体程序
PowerShell	Windows 上新的那个具体程序（功能更强，本课程用它）
shell	"外壳"的英文，泛指这一类程序

本课程统一用 PowerShell（Windows 10/11 自带，不用装）。你现在用的系统里，我之前跑的所有命令都是它。

2.3 一条命令长什么样

```
npm  run  dev
 ↑    ↑    ↑
 |    |    └─ 参数：具体要跑哪一个
 |    └────── 子命令：我要执行
 └─────────── 程序名：用哪个工具
```

空格分隔，顺序固定：**程序名 → 子命令 → 参数。**

这就像点菜："服务员（程序）→ 我要点单（子命令）→ 宫保鸡丁（参数）"。

三、在项目里看真实文件

现在把上面的概念落到你的文件夹上。

你的项目在这里：

```
C:\Users\34123\Desktop\记账簿
```

打开这个文件夹，你会看到这些（我按"你会不会碰"分了类）：

名字	类型	是什么	你会碰它吗
`src`	文件夹	**所有源代码**（记账簿的"大脑"）	会，模块 3 开始
`scripts`	文件夹	辅助脚本（打包、测试、验证）	会用，不常改
`docs`	文件夹	文档、报告、界面截图	会看
`resources`	文件夹	图标、OCR 识别模型	基本不碰
`website`	文件夹	官网	会，模块 6
`release`	文件夹	**打好的安装包**（能直接发给朋友）	会，模块 6
`分享包`	文件夹	打包好准备发出去的压缩包	会
`node_modules`	文件夹	别人写好的代码库（依赖）	**永远不要动**
`out` `build` `.tmp`	文件夹	临时产物，随时可重建	不要动
`package.json`	文件	**项目的身份证**：叫什么、依赖什么、有哪些命令	会看
`README.md`	文件	项目说明书	会看

一个重要的心理准备：`node_modules` 里有 1 万多个文件、将近 1 GB，但**没有一行是你写的，你也不需要看。**它们是你用到的"零件"——就像你开车不需要理解轮胎的橡胶配方。

四、动手任务

目标：打开终端，完成 6 个动作，最后确认你的环境。

步骤 1：打开终端

三种方法，任选：

- **方法 A（推荐）**：按 `Win` 键 → 输入 `powershell` → 回车。
- **方法 B**：按 `Win + X` → 点「终端」或「Windows PowerShell」。
- **方法 C**：在文件资源管理器里，**进到** `记账簿` **文件夹**，在地址栏输入 `powershell` 回车 —— 这样打开就直接站在这个文件夹里，很省事。

打开后你会看到一个窗口，里面有一行字，末尾是个 `>`，光标在闪。那一行叫**提示符**，它在说"我准备好了，请下命令"。

步骤 2：问它"我在哪"

输入下面这行，回车：

```
pwd
```

预期看到：一个路径，比如 `C:\Users\34123`。

`pwd` = **p**rint **w**orking **d**irectory（打印工作目录）。这是你用得最多的命令，因为**迷路时它能救你。**

步骤 3：看这里有什么

```
ls
```

预期看到：一列名字，比如 `Desktop`、`Documents`、`Downloads`。

`ls` = **l**i**s**t（列表）。如果你用的是老 CMD，这条命令叫 `dir`，效果一样。

步骤 4：走到记账簿去

```
cd "C:\Users\34123\Desktop\记账簿"
```

预期看到：提示符前面的路径变了，变成你刚输入的这个。

`cd` = **c**hange **d**irectory（换目录）。

注意两点：

1. 路径里有中文，所以**必须加引号**。不加引号时，终端可能把中文当成乱码或把空格当成"两个参数"——这就是 1.2 说的"一字不差"。
2. 路径里用的是**反斜杠** `\`（Windows 习惯），不是 `/`。

再执行一次 `pwd`，确认你真的站在这里了。

步骤 5：看看隐藏文件

```
ls -Force
```

预期看到：比刚才多出一些条目，包括 `.gitignore` 这种以点开头的文件。

`-Force` 是"参数"，意思是"把隐藏的也给我看"。

为什么重要：项目里很多关键文件是隐藏的（`.gitignore`、`.env` 等）。看不到它们，你会以为文件丢了。

步骤 6：确认环境（这一步最有用）

```
node -v
npm -v
```

预期看到：两行版本号，比如

```
v24.18.0
10.9.0
```

这两行意味着什么：

- **Node.js** 是一个"能在电脑上跑 JavaScript 的程序"。记账簿的代码要用它来运行和打包。
- **npm** 是 Node 自带的一个"零件安装器"。前面那些 `npm run ...`、`npm install` 都是它在干活，`-v` 是问它版本（version）。

如果看到"无法将 node 识别为……"：说明 Node.js 没装或没装好。这没关系，第 0-2 讲会带你装。

步骤 7（收尾）：回到上层

```
cd ..
```

`..` 表示"上一层"。执行后再 `pwd` 看一次，确认你退回去了。

卡住了怎么办

现象	原因	怎么办
`无法将"xxx"项识别为cmdlet…`	命令名打错，或这个程序没装	检查拼写；`node -v` 报错就是没装 Node
`找不到路径`	路径打错，或没加引号	把路径从文件资源管理器地址栏**复制**过来，别手打
提示符变成 `>>` 卡住	引号没配对，它在等你把话说完	按 `Ctrl + C` 退出，重新输入
中文显示成乱码	编码问题（第 1-2 讲专讲）	现在不影响操作，先继续
窗口关了，再打开位置变了	正常现象	终端不记位置，重新 `cd` 一次

五、自测题

先自己答，再看答案。

1. `pwd` 和 `ls` 分别回答什么问题？

2. 为什么这段路径必须加引号：`cd "C:\Users\34123\Desktop\记账簿"`？

3. 关掉终端再打开，你还在刚才那个文件夹吗？为什么？

4. 有人让你执行 `npm run dev`，你能说出这三部分各是什么意思吗？

5. 用一句话解释："图形界面是壳"是什么意思？

答案与解析

1. `pwd` 回答"**我在哪**"（当前所在文件夹）；`ls` 回答"**这里有什么**"（列出内容）。 → 这两个是所有终端操作的起点：先定位，再看内容。

2. 因为路径里有**中文**（和可能的空格）。终端按空格切分参数，不加引号时它可能把一部分当成参数、一部分当成别的；中文在某些编码下还会变成乱码。引号的作用是告诉它："这整段是一个整体，别拆。" → 顺带记住：**路径能从地址栏复制就绝不手打**，这是最省事的纠错方式。

3. 不在。终端**不记位置**，每次打开都从默认目录（通常是 `C:\Users\你的用户名`）开始。 → 所以每次开工第一件事往往是 `cd` 到项目目录。这也解释了为什么我给你的命令通常第一行都是 `cd`。

4. `npm` 是程序（零件安装器）；`run` 是子命令（执行脚本）；`dev` 是参数（执行 `package.json` 里名叫 `dev` 的那条脚本）。 → 你可以在项目的 `package.json` 里找到 `dev` 的定义，看看它实际做了什么（那是第 2-2 讲的内容）。

5. 意思是：你双击图标、点按钮这些操作，**只是给电脑下命令的一种包装**；真正干活的始终是底层命令与程序。图形界面让人好用，命令行让人精确——两者是同一件事的两张脸。 → 这句话理解到位，后面理解"为什么要用两个进程""为什么要打包"都会顺很多。

六、术语表

术语	一句话解释
终端 / Terminal	一个让你输入文字命令的窗口
命令行	用打字代替点鼠标的操作方式
PowerShell	Windows 上自带的现代命令行程序，本课程使用
提示符	终端里那行末尾带 `>` 的文字，表示"请下命令"
命令	一行指令，结构是「程序名 + 子命令 + 参数」
参数 / 选项	命令后面加的补充说明，如 `-Force`、`-v`
路径 / Path	一个文件或文件夹在电脑里的完整地址
`pwd`	显示当前所在位置
`ls`	列出当前位置的内容
`cd`	切换位置（change directory）
`..`	表示"上一层文件夹"
隐藏文件	名字以 `.` 开头的文件，默认不显示，要用 `ls -Force` 看
Node.js	一个能运行 JavaScript 的程序，记账簿靠它运行和打包
npm	Node 自带的"零件安装器"，负责装依赖、跑脚本
`node_modules`	所有第三方零件存放处，永远不要手动改

七、本讲与项目的对应

- 项目路径：`C:\Users\34123\Desktop\记账簿`
- 本讲不涉及代码改动，**不会影响你的任何数据**
- 下一讲：**0-2 装好你的工作台** —— 把项目真正跑起来

讲义版本：v1 ｜ 对应项目版本：0.1.0